gwflow

Purpose

`gwflow` is the cell-based 2D groundwater module that replaces the lumped aquifer (`*.aqu`) when active. It discretises the basin into a structured or unstructured mesh and solves a horizontal groundwater balance per cell, with optional couplings to channels, HRUs/LSUs, reservoirs, wetlands, floodplains, tiles, canals, solute transport, and heat transport.

Input is split across several small `*.gw` files, one concern per file, written by `swatplus-editor` from the project database and read by the `gwflow_*.f90` routines. The one exception is `gwflow.con`, the channel-cell connect file, which keeps the `.con` extension (it is a standard SWAT+ connect file listed in `file.cio`). The older monolithic `gwflow.input` master file is no longer written or required.

Activation

Set `gwflow = 1` in `codes.bsn`. The model then expects `chancell.gw` to exist. `basin_read_objs` opens it to count channel-cell intersections, sets `sp_ob%gwflow` to that count, switches `aqu_con` to `null`, and points `gwflow_con` at `gwflow.con`. If `chancell.gw` is missing, `bsn_cc%gwflow` is reset to 0 and the lumped aquifer is used.

Format conventions

- **Line 1 of every file** is a description and timestamp line written by the editor; the reader skips it.
- Most tabular files have **2 preamble lines**: the description line, then a column-header row. Data starts on line 3. (`gwflow.con` follows the SWAT+ `.con` convention: a title line, then a header row.)
- Numeric tokens are whitespace separated; the reader uses list-directed input, so exact column widths do not matter. The editor right-justifies for readability.
- Optional override columns (`strK`, `strthick`, `bc_type`, `tile_*`, `init_temp`) carry the literal `null` when unset.
- Cell IDs run `1..ncell`, one row per active cell, ascending; row position equals `cell_id`. Every file that references a cell uses that same ID space, so no `cell_id` may exceed `ncell`. For unstructured grids `row`/`col` are 0.

File overview

The writer's `write()` driver decides which files to emit.

File	Written when	Purpose
`codes.gw`	always	Feature flags and run controls (wide table)
`zones.gw`	always	Aquifer/streambed zone parameters
`cells.gw`	always	Per-cell static properties and initial state
`cellcon.gw`	always	Cell-to-cell connection (neighbour) lists
`outputs.gw`	always	Output times, observation cells, debug cell
`chancell.gw`	channel cells exist	Channel-to-cell exchange links (incl. depth-zone and observation flags)
`gwflow.con`	channel cells exist	Channel-cell spatial connections (`.con` style; keeps `.con`)
`hrucell.gw`	`conn_type` = 1 (HRU)	HRU-to-cell area mapping
`lsucell.gw`	`conn_type` = 2 (LSU)	LSU-to-cell area mapping

File	Written when	Purpose
rescell.gw	reservoir_exchange	Reservoir-cell links
floodplain.gw	floodplain_exchange	Floodplain-cell links
wetland.gw	wetland_exchange	Wetland bed thickness
tile.gw	tile_drainage	Tile drainage parameters and per-cell flag
solute.gw	solute_transport	Solute transport configuration
cell_sol.gw	solute_transport	Per-cell initial solute concentrations
pumpex.gw	pumpex, if pumps defined	External pumping schedule
hru_pump.gw	if HRUs selected	HRUs to report pumping output for

The optional flat files (tvheads.gw, ponds.gw + pond_cell.gw + pond_div.gw, phreato.gw + phreato_cell.gw, sw_group.gw, chan_depth.gw) are each written when their database table is populated. See their sections below.

The files use the *.gw extension (content-first SWAT+ naming, like plants.plt); gwflow.con is the only one that keeps .con. When a flag is off, the writer deletes any stale file.

The standalone channel-cell observation and depth-zone files from the earlier format are gone: they are now trailing columns (obs, dep_zone) of chancell.gw. The daily channel-depth series remains a separate file, chan_depth.gw.

---

codes.gw: feature flags and run controls

Wide table, same shape as codes.bsn: the description line, then a header row of 23 key names, then one value row, right-justified. The reader matches keys by name, so column order does not matter.

Keys in written order:

```
grid_type ncell cell_size n_rows n_cols bc_type conn_type gw_soil satx pumpex
tile res wet fp canal solute heat time_step write_day write_mon write_yr write_aa river_thresh
```

Key	Meaning
grid_type	structured or unstructured
ncell	Number of active cells
cell_size	Cell size, m (0 = unstructured)
n_rows	Grid rows (0 = unstructured)
n_cols	Grid cols (0 = unstructured)
bc_type	Default boundary condition (1 = const head, 2 = no flow)
conn_type	Recharge connection (1 = HRU, 2 = LSU)
gw_soil	Groundwater-to-soil transfer (0/1)
satx	Saturation-excess flow (0/1)
pumpex	External pumping (0/1)
tile	Tile drainage (0/1)
res	Reservoir exchange (0/1)
wet	Wetland exchange (0/1)
fp	Floodplain exchange (0/1)
canal	Canal seepage (0/1)
solute	Solute transport (0/1)
heat	Heat transport (0/1)
time_step	Solver time step (days)
write_day	Daily output (0/1)
write_mon	Monthly output (0/1)
write_yr	Annual output (0/1)
write_aa	Average-annual output (0/1)
river_thresh	River-bed elevation threshold (m)

Descriptions are not stored in the file; the doc-of-record for the key list is mergeWork/inputWork/gwflow_codes_reference.md, kept in sync by the editor. Heat is plumbed but off by default (heat 0).

Output frequency is hybrid: the `write_day/write_mon/write_yr/write_aa` flags here set the gwflow output frequency, but `print.prt` overrides them when it carries `gwflow_wb`, `gwflow_obs`, or `gwflow_pump` rows. With no such rows in `print.prt`, the `codes.gw` flags win.

`zones.gw`: aquifer and streambed zones

2 preamble lines. One row per zone, `zone_id` ascending. 6 columns:

#	Column	Meaning
1	`zone_id`	Zone index
2	`aquifer_K`	Aquifer hydraulic conductivity (m/day)
3	`aquifer_Sy`	Specific yield
4	`streambed_K`	Streambed hydraulic conductivity (m/day)
5	`streambed_thick`	Streambed thickness (m)
6	`thermal_K`	Thermal conductivity (heat batch; 0 when off)

`cells.gw`: cell properties and initial state

2 preamble lines. One row per active cell, `id` running `1..ncell`. 23 columns, leading with the SWAT+ object convention `id` / `name` / `gis_id`:

#	Column	Meaning
1	`id`	Cell index, 1..ncell, equal to row position
2	`name`	Cell name, `cellNNNN` (zero-padded, e.g. `cell0001`)
3	`gis_id`	Authoritative GIS cell id from `gwflowcells.shp`, equal to `id` (1..ncell)
4	`status`	Cell status code
5	`elev`	Ground-surface elevation (m)
6	`thck`	Aquifer thickness (m)
7	`K_zone`	Aquifer K zone id
8	`Sy_zone`	Specific-yield zone id
9	`delay`	Recharge delay (days)
10	`exdp`	ET extinction depth (m)
11	`init`	Initial groundwater head (m)
12	`x`	Cell-centre x coordinate
13	`y`	Cell-centre y coordinate
14	`area`	Cell area (m^2)
15	`strK`	Streambed K override (`null` if unset)
16	`strthick`	Streambed thickness override (`null` if unset)
17	`bc_type`	Per-cell boundary condition override (`null` if unset)
18	`tile_depth`	Tile depth override (`null` if unset)
19	`tile_area`	Tile area override (`null` if unset)
20	`tile_K`	Tile K override (`null` if unset)
21	`row`	Structured grid row (0 if unstructured)
22	`col`	Structured grid col (0 if unstructured)
23	`init_temp`	Initial groundwater temperature (`null` when heat off)

`gis_id` is the cell id carried straight from the QSWAT+ `gwflowcells.shp` layer (it equals `id`, since the shapefile numbers active cells `1..ncell`). It is used for the `gis_id` output column and for cell and pump naming, replacing an earlier row/col computation. The reader consumes the columns by position, so column order is fixed.

`cellcon.gw`: cell-to-cell connections

2 preamble lines (description, then `cell_id  neighbor_tot  neighbor_ids`). One variable-width row per cell `1..ncell`:

- `cell_id`
- `neighbor_tot` (neighbour count)

- then `neighbor_tot` neighbour `cell_id`s

This defines the solver stencil (which cells are adjacent).

`outputs.gw`: output control

2 preamble lines (description, then `category  value  note`). One row per item:

`category`	`value`
`head_output_time`	output time, `YYYYDDD` (one row per requested time)
`observation_cell`	`cell_id` of an observation well (one row each)
`detail_debug_cell`	a `cell_id`, or `0` for none

`chancell.gw`: channel-to-cell links

2 preamble lines (description, then column header). One row per channel-cell link. 7 columns:

#	Column	Meaning
1	`cell_id`	Cell exchanging with the channel (1..ncell)
2	`elev_m`	Channel-bed elevation (m)
3	`channel`	Channel id (matches `chandeg.con`)
4	`riv_length_m`	Channel length within the cell (m)
5	`zone`	Depth-zone index
6	`dep_zone`	Channel depth-zone index, links the cell to its series in `chan_depth.gw`
7	`obs`	1 to report gw-channel exchange output for this cell, else 0

A cell may appear on multiple rows when more than one channel crosses it. The `dep_zone` and `obs` columns fold in two files from the earlier format (the separate channel-depth-zone file and the channel-observation file).

`gwflow.con`: channel-cell spatial connections

SWAT+ `.con`-style file, keeping the `.con` extension. Line 1 is the title `gwflow.con: channel-cell spatial connections`; line 2 is the `.con` header:

```
NUMB NAME GISID AREA LAT LONG ELEV CELL WST CONST OVER RULE SRC_TOT OBTYPE_OUT1 OBTYPNO_OUT1 HTYPE_OUT1 FR
```

One row per channel cell. `CELL` is the gwflow `cell_id`, `OBTYPNO_OUT1` is the channel connect index, with fixed `sdc` / `tot` / `1.00` in the type/total/fraction fields.

`hrucell.gw`: HRU-to-cell mapping

2 preamble lines (description, then column header). One row per HRU-cell overlap. 4 columns:

#	Column	Meaning
1	`hru`	HRU id
2	`area_m2`	HRU area (m^2)
3	`cell_id`	Cell overlapping the HRU
4	`overlap_m2`	Overlap area (m^2)

`lsucell.gw` is the LSU analogue, written when `conn_type` is 2.

`wetland.gw`: wetland bed thickness

2 preamble lines (description, then `name  bed_thick`). One row per wetland: wetland `name`, bed thickness (m).

hru_pump.gw: HRU pump-observation list

Selector list of HRUs to report pumping output for. Existence-gated: the editor writes it when the source table has rows, and removes it otherwise; no codes.gw flag. Meta line, then a count, then one hru_id per line (ascending).

Other conditional files

Written only when the matching codes.gw flag is on. These keep a title/description line then a blank line before data (no column-header row):

- lsucell.gw: LSU-to-cell mapping, one row per overlap: lsu_id cell_id area_m2.
- rescell.gw: reservoir-cell links: bed thickness and bed K, count, then cell_id res_id res_stage. (Not present when no reservoir cells.)
- floodplain.gw: floodplain-cell links: count, then cell_id channel_id K area_m2. (Not present when no floodplain cells.)
- tile.gw: tile drainage: a defaults row tile_depth tile_area tile_k, then a tile-group count, then a per-cell cell_id list.
- solute.gw: solute transport config: transport steps, dispersion coefficient, solute count nsolute, then one row per solute name sorption_coef rate_const init_conc_aquifer init_conc_recharge.

cell_sol.gw: per-cell initial solute concentrations

Written when solute transport is on. 2 preamble lines (description, then cell_id conc_1 … conc_nsolute). One row per cell 1..ncell, with one concentration column per solute in solute.gw order:

```
cell_sol.gw: ...
 cell_id        conc_1        conc_2   ...   conc_10
       1      12.50000       0.00000   ...   0.00000
       2       7.70000       0.00000   ...   0.00000
```

New flat optional files

These optional files were harmonized to the flat convention: a description line, a header row, then rows to end-of-file with no embedded count lines. Each is existence-gated, written from its database table when populated. The formats below match the editor's on-disk output. The deferred items minerals.gw (salt-mineral parameters) and transit.gw (transit-time tracking) are reader-supported but not exported by the editor.

tvheads.gw: time-varying boundary heads

One row per boundary cell: cell_id then one head value per simulation year (positional, 1..Nbyr; the reader takes Nbyr from the run period, so the file has no count line). A 0 means no specified head that year.

```
tvheads.gw: ...
   cell_id      head_yr1      head_yr2      head_yr3      head_yr4      head_yr5      head_yr6
         1       0.00000      12.50000       0.00000       9.00000       0.00000       0.00000
         5       0.00000       0.00000       7.00000       0.00000       0.00000       4.40000
```

ponds.gw + pond_cell.gw: recharge ponds

ponds.gw is one row per pond: 11 fixed columns then one unl_conc per solute (unl_conc_1 … unl_conc_Nsolute, unsaturated-zone concentration of each solute):

```
ponds.gw: ...
 pond_id  area  chan  canal  unl  bed_k  wsta  evap_co  start_yr  start_mo  start_day  unl_conc_1..unl_con
       1   25000.00   5   0   1   0.000100   3   0.70000   1981   4   1    2.50  0.30  ...
```

Column	Meaning
pond_id	Pond index
area	Pond area (m²)
chan	Target channel id

Column	Meaning
`canal`	Target canal id
`unl`	Unsaturated-zone flag (0/1)
`bed_k`	Pond-bed hydraulic conductivity (m/day)
`wsta`	Weather-station id
`evap_co`	Evaporation coefficient
`start_yr` / `start_mo` / `start_day`	Divert start date
`unl_conc_1..N`	Unsaturated-zone concentration per solute

`pond_cell.gw` lists pond-to-cell connections, one per row: `pond_id  cell_id  conn_area` (m^2). The number of connection rows defines the per-pond cell count; there is no count line.

`pond_div.gw`: pond daily diversion series

Daily diversion volume into each pond. Dense: one row per simulation day (0-filled on days with no diversion). The columns after the date are one `div_pondN` per pond, in `ponds.gw` pond-id order:

```
pond_div.gw: ...
    year   month     day     div_pond1     div_pond2
    1980       1       1       0.00000       0.00000
    1980       1       2       0.00000       0.00000
```

- `year` / `month` / `day`: calendar date of the row. The reader keys on row order (positional), so every simulation day must be present, including leap days.
- `div_pond1 … div_pondN`: diversion volume into each pond (m^3) that day.
- Existence-gated: written when pond diversions are defined; if absent, diversion is 0.

`sw_group.gw`: channel-cell groups

Groups of channel cells for aggregated reach gw-channel exchange output. One variable-width row per group:

```
sw_group.gw: channel-cell groups
   group_id   ncell   cell_ids
   1   2   42   43
   2   2   44   45
```

- `group_id`, then `ncell` (cells in the group), then `ncell` cell ids (1..ncell space).

`phreato.gw` + `phreato_cell.gw`: phreatophyte ET

Split into a depth-ET curve and the cells it applies to.

`phreato.gw` is the depth-rate curve, one row per point:

```
phreato.gw: depth-ET curve
     depth_m     et_rate_m
   0.0   0.005
   1.0   0.003
   2.0   0.001
```

`phreato_cell.gw` lists the cells, one per row:

```
phreato_cell.gw: phreatophyte cells
   cell_id   area_m2
   22   25000000.0
   23   25000000.0
```

`chan_depth.gw`: daily channel depth per zone

Per-zone daily channel-depth series, one row per simulation day. Wide: `jday  yr` then one depth column per channel depth zone (N = number of `dep_zone` values in `chancell.gw`):

```
chan_depth.gw: daily channel depth per zone
   jday   yr   depth_z1   depth_z2
   1   1980   0.50   0.80
   2   1980   0.50   0.80
```

pumpex.gw format

External pumping schedule, enabled by `pumpex = 1` in `codes.gw`. 2 preamble lines (meta line, then a column header), then one row per pump-period. 7 columns:

```
gwflow.pumpex: external groundwater pumping
      name    cell_id        rate   yr_start   dy_start     yr_end     dy_end
  cell0700        700      150.00       1981          1       1981        180
  cell0700        700       80.00       1981        181       1981        365
  cell0912        912      250.00       1982         90       1984        270
```

#	Column	Meaning
1	`name`	Cell name (`cellNNNN`, matching `cells.gw` col 2); label only, the reader keys on `cell_id`
2	`cell_id`	Pumped cell (1..ncell)
3	`rate`	Pumping rate (m^3/day)
4	`yr_start`	Start year
5	`dy_start`	Start day-of-year (1..365)
6	`yr_end`	End year
7	`dy_end`	End day-of-year (1..365)

- One row per pump-period; rows for the same cell are consecutive, and consecutive same-cell rows form one pump.
- Dates are calendar (year + day-of-year). The reader converts each to a simulation day-count as `(year - sim_start_year) * 365 + day_of_year`, the same 365-day convention as `ponds.gw`.

The editor writes this file (`write_pumpex`) when `pumpex = 1` and pumps are defined in the project database, one row per period straight from the database. It is skipped when no pumps exist.

Example

A 2-zone structured grid (excerpts from a generated project; values are illustrative).

codes.gw (wide, abbreviated)

```
grid_type ncell cell_size n_rows n_cols ... solute heat time_step write_aa river_thresh
structured 1454   5000.00     52     53 ...      0    0      1.00        1         5.00
```

zones.gw

```
   zone_id     aquifer_K    aquifer_Sy   streambed_K streambed_thick
         1       0.50000       0.20000       0.00500         0.50000
         2       2.00000       0.20000       0.00500         0.50000
```

cells.gw (abbreviated)

```
 id  name      gis_id  status   elev   thck K_zone Sy_zone ...  row  col init_temp
  1  cell0001       1       2 395.00  34.99      1       1 ...   52    1      null
```

cellcon.gw

```
 cell_id  neighbor_tot  neighbor_ids
       1             3       2      20      21
```

chancell.gw

```
 cell_id   elev_m  channel  riv_length_m  zone  dep_zone  obs
      42   326.00       66       1853.55     1         1    1
```

Related

- codes.bsn: `gwflow = 1` activates the module.
- object.cnt: `sp_ob%gwflow` is set from the channel-cell count and replaces `sp_ob%aqu`.
- `aquifer.aqu`, `initial.aqu`: not used when gwflow is active.